PDE4444 — MACHINE LEARNING FOR ENGINEERS

ML-Based Visual Quality Inspection System

Classifying breadboard circuits as PASS/FAIL using deep learning and classical ML — a comparative study

BEST RESULT	MODELS EVALUATED	DATASET
93.75%	**9**	**864**
CNN Tuned Validation Accuracy	CNN + 6 Classical + Variants	Augmented Images

ENGINEERING PROBLEM DEFINITION

Automated Breadboard Quality Control

- **Manual inspection** of breadboard circuits is slow, inconsistent, and error-prone — defective boards reaching downstream processes cause costly failures
- **Inputs:** Overhead RGB camera images (224×224 px) of assembled breadboard circuits
- **Output:** Binary classification — PASS or FAIL — with a confidence score, in real time
- **Deployment:** Live webcam inference via `analyzer.py`, with LED indicator and optional robotic arm integration

FAIL

Critical QC error — defective board ships

PASS

Board meets quality threshold — cleared

ENGINEERING RELEVANCE

Demonstrates a complete MLOps pipeline applicable to any visual quality inspection task in manufacturing

DATASET COLLECTION & FEATURE REPRESENTATION

Data Pipeline & Augmentation Strategy

RAW DATASET
54 Images

19 PASS · 35 FAIL

Manually captured overhead shots

→

AUGMENTATION
16× Expansion

8 rotations × 2 mirrors

offline_augmentation.py

→

AUGMENTED DATASET
864 Images

304 PASS · 560 FAIL

Class ratio preserved ~35/65

→

CNN BRANCH
224×224×3

150,528 input dims

70 / 15 / 15 split

→

CLASSICAL ML BRANCH
64×64×3

12,288 input dims

Flattened pixel vector

DIMENSIONALITY REDUCTION

PCA reduced classical ML input to **50 components** — capturing **87.1%** of variance, enabling Newton's method optimisation

PREPROCESSING

CNN: Keras `Rescaling(÷255)` layer. Classical ML: `StandardScaler` normalisation before all classifiers

CLASS IMBALANCE

35% PASS / 65% FAIL distribution preserved through augmentation — reflects real-world defect prevalence in QC scenarios

NEURAL NETWORK DESIGN

Convolutional Neural Network Architecture

INPUT	224 × 224 × 3 — RGB image
RESCALE	÷255 normalisation
CONV BLOCK 1	Conv2D(32, 3×3, ReLU) + MaxPool(2×2)
CONV BLOCK 2	Conv2D(64, 3×3, ReLU) + MaxPool(2×2)
CONV BLOCK 3	Conv2D(128, 3×3, ReLU) + MaxPool(2×2)
FLATTEN	Feature map → 1D vector
DENSE	Dense(128, ReLU) + Dropout(0.5)
OUTPUT	Dense(2, Softmax) — PASS / FAIL

Training Configuration

Optimiser: **Adam** (lr = 0.001)

Loss: **Sparse Categorical CE**

Early Stopping: **patience = 3**

Split: **70 / 15 / 15**

Baseline Performance

87.02%

Test accuracy · F1 = 0.86

FAIL recall = **95%**

NEURAL NETWORK OPTIMISATION — ACTIVATION FUNCTIONS

ReLU vs. Tanh — Activation Analysis

WINNER

91.41%

ReLU — Rectified Linear Unit

$f(x) = \max(0, x)$

Avoids vanishing gradients in deeper layers. Sparse activation supports efficient feature learning in 3-conv architecture.

UNDERPERFORMED

65.63%

Tanh — Hyperbolic Tangent

$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$

Converged to **majority-class baseline** — failed to learn discriminative features. Vanishing gradients in the 3-conv stack saturated weights, preventing meaningful updates.

+25 percentage points

ReLU outperformed Tanh by a 25.78pp margin — a decisive gap confirming that ReLU's resistance to vanishing gradients is critical even in a shallow 3-block CNN

NEURAL NETWORK OPTIMISATION — ALGORITHMS

Optimisation Algorithm Comparison

DEEP LEARNING — CNN

Adam's adaptive per-parameter learning rates handled sparse gradients and class imbalance far better than fixed-step SGD

TRADITIONAL ML — LOGISTIC REG ON PCA-50

GD diverged at lr=0.01. Newton's converged in **10 iterations** via Hessian-scaled steps — feasible because PCA reduced dims to 50

HYPERPARAMETER TUNING

Keras Tuner RandomSearch — Best CNN Configuration

BASELINE → TUNED

Before

87.02% →

After (val)

93.75% +6.73pp

SEARCH SPACE

HYPERPARAMETER	BEST VALUE
Filters (Conv1)	64
Dense Units	128
Dropout Rate	0.2
Learning Rate	0.001

Tuning Strategy

Keras Tuner RandomSearch

10 trials · 3-fold evaluation

~50–60 min on CPU

SVM Tuning — No Improvement

sklearn **RandomizedSearchCV**

10 iterations · 3-fold

Best: C=0.1, linear kernel

Result: **82.08%** (no gain — already optimal)

TAKEAWAY

Tuning helps most when capacity is underutilised — SVM was already at its ceiling

BASELINE COMPARISON — CLASSICAL ML

Six Classical Models — Ranked Performance

Best Classical Model

SVM Linear

80/20 held-out split · StandardScaler

Worst — MLP Instability

CV Std Dev: **±13.26%**

Fold 2 dropped to **34.8%** — below majority-class baseline, confirming training divergence

All models trained on flattened **12,288-dim** pixel vectors — no hand-crafted features, demonstrating raw pixel viability for classical QC

EXPERIMENTAL RIGOR

Cross-Validation & Overfitting Analysis

5-FOLD STRATIFIED CV — CLASSICAL MODELS

MODEL	CV MEAN	STD DEV
SVM (Linear)	86.98%	±2.77%
Logistic Regression	83.21%	±2.64%
Gradient Boosting	79.59%	±3.52%
Random Forest	77.42%	±1.29%
KNN (k=5)	72.36%	±2.19%
MLP	60.49%	±13.26%

SVM's CV mean (86.98%) exceeds its held-out test score (82.08%) — confirming robust generalisation, not overfitting to the split

CNN OVERFITTING ANALYSIS

Train Accuracy

99.17%

Val Accuracy

90.62%

OVERFITTING GAP

8.55pp

Dropout(0.5) partially mitigated — early stopping (patience=3) limited further divergence

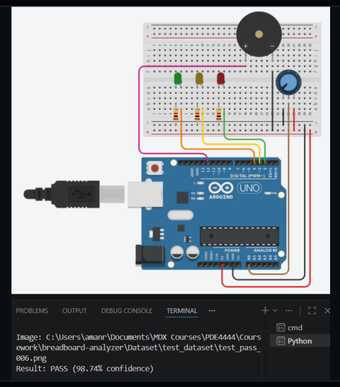
False Positives (Critical QC Error)

4 / 75 FAIL boards misclassified as PASS

FAIL recall = **95%** · 13/56 PASS boards rejected · 131 held-out images

LIVE INFERENCE — ANALYZER.PY · STATIC IMAGE MODE

Classification on Unseen Data

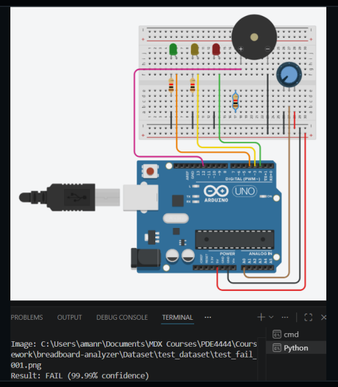


CORRECT — TRUE PASS

PASS

Confidence: **98.74%**

All components correctly seated

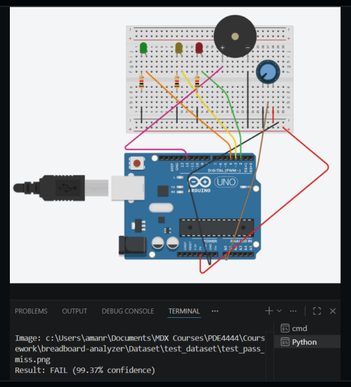


CORRECT — TRUE FAIL

FAIL

Confidence: **99.99%**

Moved component detected via learned feature maps



MISCLASSIFICATION — FALSE FAIL

PASS → FAIL

Confidence: **99.37%**

Unclear wiring detected as improper breadboard design

HOW IT WAS RUN

`python analyzer.py` → select option **1** (static image) → provide image path → result printed to terminal with class label and confidence score

FINAL RESULTS & DEPLOYMENT

Model Comparison & System Architecture

ALL MODELS — FINAL RANKING

MODEL	ACCURACY	NOTES
CNN — Tuned (RandomSearch)	**93.75%**	**Best overall**
CNN — ReLU + Adam	87.02%	Baseline CNN
SVM (Linear) — 5-fold CV	86.98%	Best classical
SVM (Linear) — held-out	82.08%	—
Logistic Regression	79.19%	—
Gradient Boosting	78.03%	—
Random Forest	74.57%	—
KNN (k=5)	69.94%	—
MLP (sklearn)	64.74%	Unstable

Conclusion

CNN surpasses the best classical model by **+4.94pp** on test data and **+6.73pp** after tuning — validating deep learning for visual QC tasks with a small augmented dataset

DEPLOYMENT STACK

- `analyzer.py` — Webcam or static image inference, real-time CV overlay
- Arduino LED indicator — Green/Red output on PASS/FAIL
- EPSON VT6 robotic arm integration (`picker-bot`) for automated sorting

FUTURE WORK | Transfer learning · expanded dataset · Raspberry Pi edge deployment